

OPERATING SYSTEM

UNIT-2

PRAVIN S. GAME, PH.D.

ASSOCIATE PROFESSOR

DEPARTMENT OF COMPUTER ENGINEERING

PIMPRI CHINCHWAD COLLEGE OF ENGINEERING

•Course Objectives:

1. To introduce the Operating system and process, threads and CPU scheduling
2. To explore inter process communication mechanisms, to introduce the critical-section problem and solutions.
3. To explore various techniques of allocating memory to processes and explain the concepts of demand paging, page-replacement algorithms.
4. To describe the details of implementing local file systems and directory structures

•Course Outcomes:

After learning the course, the students should be able to:

1. Compare various process scheduling algorithms for a given snapshot of the system.
2. Analyze IPC mechanisms and solutions of process synchronization for critical section problems.
3. Analyze the performance of memory management algorithms for a given problem.
4. Apply disk scheduling policies for a given I/O request sequence with file management concepts

PRAVIN S. GAME

UNIT - 2

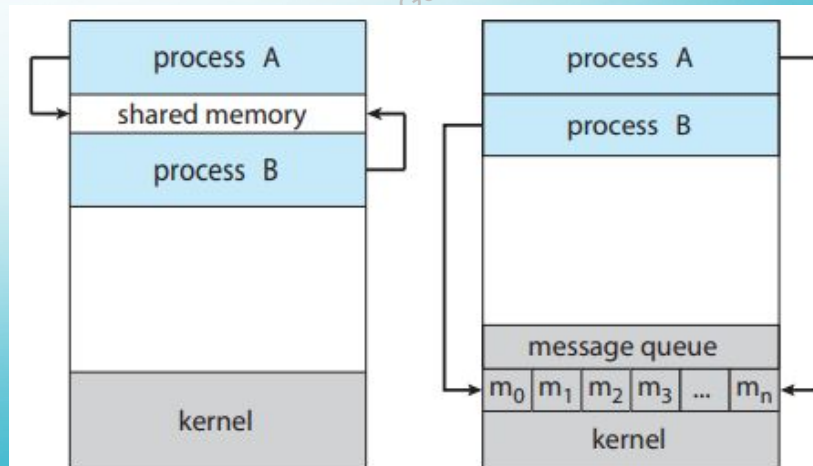
•IPC and Process Synchronization:

Inter process Communication mechanisms: Pipes, Shared memory, Message passing

Process Synchronization : Introduction, Critical-Section Problem, Hardware Support for Synchronization, Mutex Locks, Semaphores, Synchronization problem: Reader-writer, producer- consumer problem, Dining Philosophers problem

PRAVIN S. GAME

- Processes executing concurrently may be either
 - Independent** : no shared data
 - Cooperating** : shares data; is affected and can affect other processes
- Why do processes cooperate?
 - Information sharing (e.g. copy-paste)
 - Computation speedup (parallelization)
 - Modularity
- So, they need IPC
- Two models of IPC
 - Shared Memory
 - Message-passing



PRAVIN S. GAME

Shared Memory	Message Passing
Shares a memory region for communication	Maintains message queue
Useful for sharing large data	Useful for small data exchange
Difficult to implement in distributed systems	Easier to implement in distributed system
Faster than message passing	Slower than shared memory as it uses system calls
No Kernel in intervention	As system calls are used, kernel intervention occurs
Takes less time	Time-consuming due to kernel intervention

PRAVIN S. GAME

SHARED MEMORY MECHANISM

- Share a memory region
- Shared region resides in memory of process creating shared memory segment
- Other processes attach this region to their address space
- Processes agree to override OS restriction ?
- Data and location decided by processes not by OS

COOPERATION

- Producer-Consumer problem

PRAVIN S. GAME

- Solution for Producer-Consumer problem : Shared Memory

- **Buffer** - residing in shared memory region

- Producer produces an item while consumer is consuming another : synchronize
- Unbounded and Bounded buffer

- Implemented as a circular array with two pointers

- Next free location – in pointer

- First full position – out pointer

```
#define BUFFER_SIZE 10
typedef struct {
    ...
} item;
item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```

PRAVIN S. GAME

****Producer****

```
while (true) {  
    /* Produce an item */  
    while (((in = (in + 1) %  
        BUFFER_SIZE count) == out) ;  
    /* do nothing -- no free buffers */  
    buffer[in] = item;  
    in = (in + 1) % BUFFER_SIZE;  
}
```

****Consumer****

```
while (true) {  
    while (in == out)  
        // do nothing -- nothing to consume  
    // remove an item from the buffer  
    item = buffer[out];  
    out = (out + 1) % BUFFER_SIZE;  
    return item;  
}
```

PRAVIN S. GAME

PIPES

- Acts as a conduit allowing two processes to communicate

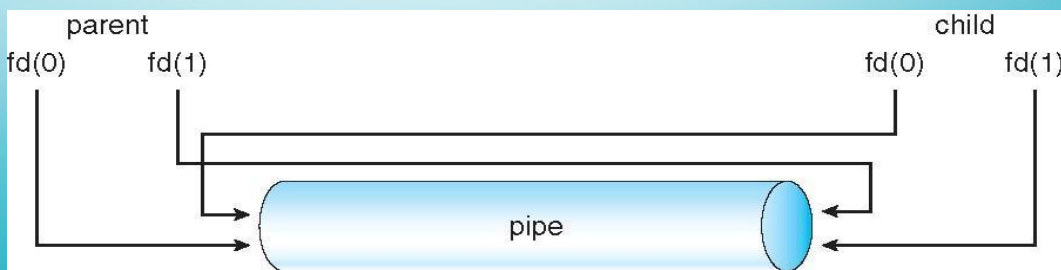
•Issues

- Is communication unidirectional or bidirectional?
- In the case of two-way communication, is it half or full-duplex?
- Must there exist a relationship (i.e. parent-child) between the communicating processes?
- Can the pipes be used over a network?

PRAVIN S. GAME

Ordinary Pipes

- **Ordinary Pipes** allow communication in standard producer-consumer style
- Producer writes to one end (the *write-end* of the pipe)
- Consumer reads from the other end (the *read-end* of the pipe)
- Ordinary pipes are therefore unidirectional
- Require parent-child relationship between communicating processes



Named Pipes

- Named Pipes are more powerful than ordinary pipes
- Communication is bidirectional
- No parent-child relationship is necessary between the communicating processes
- Several processes can use the named pipe for communication
- Provided on both UNIX and Windows systems

IPC- MESSAGE PASSING

- No shared memory region
- Message size- fixed or variable
- Operations:
 - **send**(message)
 - **receive**(message)
- Process P and Q wish to communicate?
 - establish a **communication link** between them
 - exchange messages via send/receive
- Communication:
 - Direct
 - In-direct

PRAVIN S. GAME

Direct Communication	In-direct Communication
Processes must name each other explicitly	Messages are directed and received from mailboxes
Links are established automatically	Link established only if processes share a common mailbox
A link is associated with exactly one pair of communicating processes	A link may be associated with many processes
Between each pair there exists exactly one link	Each pair of processes may share several communication links
The link may be unidirectional, but is usually bi-directional	Link may be unidirectional or bi-directional

PRAVIN S. GAME

- Message passing may be either blocking (synchronous) or non-blocking (asynchronous)
- **Synchronous**
 - **Blocking send** has the sender block until the message is received
 - **Blocking receive** has the receiver block until a message is available
- **Asynchronous**
 - **Non-blocking send** has the sender send the message and continue
 - **Non-blocking receive** has the receiver receive a valid message or null

PRAVIN S. GAME

PROCESS SYNCHRONIZATION

PRAVIN S. GAME

INTRODUCTION

- Cooperating processes
- Concurrent access to shared data may result in data inconsistency
- Maintaining data consistency requires mechanisms to ensure the orderly execution of cooperating processes
- Producer-consumer problem (modified code)

PRAVIN S. GAME

```
/*Producer*/
```

```
while (true)
```

```
{ /* produce an item in next_ produced */
```

```
    while (count==BUFFER_SIZE)
```

```
        ; /* do nothing */
```

```
    buffer[in] = next_produced;
```

```
    in = (in + 1) % BUFFER_SIZE;
```

```
    count++;
```

```
}
```

```
/*Consumer*/
```

```
while (true)
```

```
{
```

```
    while (count == 0)
```

```
        ; /* do nothing */
```

```
    next_Consumed = buffer[out] ;
```

```
    out = (out + 1) % BUFFER_SIZE;
```

```
    count--;
```

```
}
```

Independently correct. Concurrently ??

PRAVIN S. GAME

- Assume count = 5
- Concurrent execution of count++ and count-- : 4, 5, or 6?
- Implementation in Machine language (register 1 and 2 are local CPU registers)

count++ :

```
register1 = count
register1 = register1 + 1
count = register1
```

count-- :

```
register2 = count
register2 = register2 - 1
count = register2
```

- There could be different sequence of executions of these statements

PRAVIN S. GAME

- T0: producer execute register1 = count {register1 = 5}
- T1: producer execute register1 = register1 + 1 {register1 = 6}
- T2: consumer execute register2 = count {register2 = 5}
- T3: consumer execute register2 = register2 - 1 {register2 = 4}
- T4: consumer execute count = register2 {count = 4}
- T5: producer execute count = register1 {count = 6}

- Incorrect answer – b'coz both processes are allowed to manipulate concurrently

□ Race Condition

- Solution : **Process Synchronization**

PRAVIN S. GAME

CRITICAL SECTION

- Processes share data
- Section of the code with this shared data – Critical section
- It must be protected
- Each process has **critical section** segment of code
 - Process may be changing common variables, updating table, writing file, etc
 - When one process in critical section, no other may be in its critical section
- Critical section problem is to design protocol to solve this
- Each process must ask permission to enter critical section in **entry section**, may follow critical section with **exit section**, then **remainder section**
- Especially challenging with preemptive kernels

PRAVIN S. GAME

Solution to Critical Section problem requires :

Step 1: Mutual exclusion

Step 2: Progress

Step 3: Bounded waiting

```
while (true) {  
    entry section  
    critical section  
    exit section  
    remainder section  
}
```

General structure of a process

PRAVIN S. GAME

CRITICAL SECTION PROBLEM

- Consider system of n processes $\{p_0, p_1, \dots, p_{n-1}\}$
- Each process has **critical section** segment of code
 - Process may be changing common variables, updating table, writing file, etc
 - When one process in critical section, no other may be in its critical section
- Critical section problem is to design protocol to solve this
- Each process must ask permission to enter critical section in **entry section**, may follow critical section with **exit section**, then **remainder section**
- Especially challenging with preemptive kernels

PRAVIN S. GAME

General approaches to handle critical-section problem:

- Preemptive kernels
 - allows a process to be preempted while it is running in kernel mode
 - may be more responsive
 - more suitable for real-time programming
- Nonpreemptive kernels
 - does not allow a process running in kernel mode to be preempted

PRAVIN S. GAME

HARDWARE SUPPORT

- Many systems provide hardware support for critical section code
- Uniprocessors – could disable interrupts
 - Currently running code would execute without preemption
 - Generally too inefficient on multiprocessor systems
 - Operating systems using this are not broadly scalable
- Three ways
 - Memory Barriers
 - Hardware Instruction : test memory word and set value Or swap contents of two memory words
 - Atomic = non-interruptable

PRAVIN S. GAME

MEMORY BARRIERS

- A system may reorder instructions – inconsistency
- Memory model:
 - Determination of memory guarantee by a computer architecture to an application program
- Two types of Memory models:
 - Strongly ordered
 - Weakly ordered
- Memory models vary by processor type --- impacts kernel development
- Addressed by memory barriers
- System ensures that all loads and stores are completed before any subsequent load or store operations are performed

PRAVIN S. GAME

while (!flag)
 memory_barrier();
print x;

Guarantee: the value of flag is loaded before the value of x

PRAVIN S. GAME

HARDWARE INSTRUCTIONS

- Test and set definition

```
boolean test_and_set(boolean *target)
{
    boolean rv = *target;
    *target = true;
    return rv;
}
```

- ME with Test and set

```
do {
    while (test_and_set(&lock)) ;
    /* do nothing */
    /*when available enter critical section */
    lock = false;
    /* remainder section */
} while (true);
```

PRAVIN S. GAME

COMPARE AND SWAP (CAS)

- Mutual exclusion with compare-and-swap
- Atomic execution
- To use it for ME call cas()
 - Global variable *lock* initialised to 0
 - A process invoking cas(), will set *lock* to 1
 - Enters critical section as original value of *lock* is equal to expected value 0.
 - After exiting critical section, *lock* is set back to 0

```
int cas(int *value, int expected,
int new_value)
{
    int temp = *value;
    if (*value == expected)
        *value = new_value;
    return temp;
}
```

```
while (true)
{ while (cas(&lock, 0, 1) != 0)
    ; /* do nothing */
    /* critical section */
    lock = 0;
    /* remainder section */
}
```

```
while (true) {
    waiting[i] = true;
    key = 1;
    while (waiting[i] && key == 1)
        key = compare_and_swap(&lock, 0, 1);
    waiting[i] = false;
    /* critical section */
    j = (i + 1) % n;
    while ((j != i) && !waiting[j])
        j = (j + 1) % n;
    if (j == i)
        lock = 0;
    else
        waiting[j] = false;
    /* remainder section */
}
```

Bounded-waiting
mutual exclusion with
compare and swap()

Atomic Variable

- CAS is not used directly to provide ME
- CAS acts as building block
- **Atomic** variable
- increment (&count)

```
void increment(atomic int *v)
{
    int temp;
    do {
        temp = *v;
    }
    while (temp != compare and swap(v,
        temp, temp+1));
}
```

PRAVIN S. GAME

MUTEX LOCKS

- Hardware based solution – complex and inaccessible to application programs
- Software solutions at higher levels : Mutex locks - Mutual Exclusion
- Protects critical section and prevents race conditions
- acquire(), release() functions
- A Boolean variable 'available' to check availability of lock
- Calls to either acquire() or release() must be *Atomic*
- Can be implemented using CAS
- Contended or Uncontended locks

```
while (true) {
```

```
    acquire lock
```

```
        critical section
```

```
    release lock
```

```
        remainder section
```

```
}
```

PRAVIN S. GAME

- The implementation requires **busy waiting**.
- While a process is in its critical section, any other process that tries to enter its critical section must loop continuously in the call to acquire()
- Such mutex locks aka spinlocks
- No Context Switch is required when a process must wait on a lock

PRAVIN S. GAME

SEMAPHORES

- A semaphore S is an integer variable
- Accesses through two standard atomic operations:
 - wait() and signal().
- Originally, P() and V()
- P - proberen, to test
- V- verhogen, to increment
- one process modifies the semaphore value, no other process can simultaneously modify that same semaphore value — Atomic

```
wait(S) {
    while (S <= 0)
        ; // busy wait
    S--;
}
```

```
signal(S) {
    S++;
}
```

PRAVIN S. GAME

- Counting semaphore – value can range over unrestricted domain
- Binary semaphore - 0 and 1
- Counting semaphore useful for access control to resource with finite instances
- Busy waiting problem
 - Avoided by suspending the process executing wait() operation
 - Process placed in semaphore's waiting queue
 - CPU scheduler can schedule next process
 - wakeup() operation is used to place the process in ready queue
- Implementation (next slide)

PRAVIN S. GAME

- Implementation of wait:

```
wait(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        add this process to S->list;
        block();
    }
}
```

```
typedef struct {
    int value;
    struct process *list;
} semaphore;
```

- Implementation of signal:

```
signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) {
        remove a process P from S->list;
        wakeup(P);
    }
}
```

PRAVIN S. GAME

DEADLOCK AND STARVATION

- **Deadlock** – two or more processes are waiting indefinitely for an event that can be caused by only one of the waiting processes
- Let **S** and **Q** be two semaphores initialized to 1

P_0	P_1
wait (S);	wait (Q);
wait (Q);	wait (S);
...	...
signal (S);	signal (Q);
signal (Q);	signal (S);

- **Starvation** – indefinite blocking
 - A process may never be removed from the semaphore queue in which it is suspended
- **Priority Inversion** – Scheduling problem when lower-priority process holds a lock needed by higher-priority process
 - Solved via **priority-inheritance protocol**

PRAVIN S. GAME

READERS-WRITERS PROBLEM

- A shared database – with concurrent processes – possibilities?

- Readers and Writers

- Cases

- Two or more readers
- One writer and one or more readers
- Two or more writers

- READERS–WRITERS problem

- Solution :

- allow multiple readers to read shared data simultaneously
- Only one writer to access shared data at a time

-RR
-RW
-WW
-WR

PRAVIN S. GAME

- Variations (involve priorities)

- First readers-writers problem: no reader be kept waiting unless a writer has already obtained permission to use the shared
- Second readers-writers problem: once a writer is ready, that writer perform its write as soon as possible

- Both cause starvation

- Solution to *First* problem:

- Reader processes share following data structures:

```
semaphore rw_mutex = 1; //binary
semaphore mutex = 1;
int read_count = 0;    //counting
```

PRAVIN S. GAME

- rw_mutex:

- Common to both R & W
- Provides ME for writers
- Used by the first or last reader that enters or exits the critical section.
- Not used by readers that enter or exit while other readers are in their critical sections

- mutex :

- ensures ME when read_count is updated

- read_count :

- Counting semaphore
- Keeps track of no. of reading processes


```

while (true)
{
    wait(rw_mutex);
    ...
    /* writing is performed */
    ...
    signal(rw_mutex);
}

```

Writer Process

PRAVIN S. GAME

```

while (true) {
    wait(mutex);
    read_count++;
    if (read_count == 1)
        wait(rw_mutex);
    signal(mutex);
    ...
    /* reading is performed */
    ...
    wait(mutex);
    read count--;
    if (read_count == 0)
        signal(rw_mutex);
    signal(mutex);
}

```

If writer is in critical section,
One waiting reader queued
on rw_mutex,
all others on mutex

Once writer executes
signal(rw_mutex),
scheduler will allow
readers or one writer

Reader Process

PRAVIN S. GAME

- Solutions have been generalized to provide reader-writer locks

- Acquire R-W lock by specifying mode of access- read or write

- Application

- Where identification reader and writer processes is possible.

- Where there are more readers than writers

PRAVIN S. GAME

Producer-Consumer Problem

- Bounded buffer case....

- Producer should not add item in a full buffer
- Consumer should not remove an item from empty buffer
- Multiple producers and/or consumers should not access buffer simultaneously.

Critical section ???

Let's see the problem....

PRAVIN S. GAME

```
import threading
```

```
# Our shared buffer counter -- Critical section  
counter = 0
```

```
# We use a large number to force the CPU to switch contexts  
# while the threads are running.  
ITERATIONS = 1000000
```

```
def producer():  
    global counter  
    for _ in range(ITERATIONS):  
        # This looks like one step, but internally three steps.. Remember!  
        counter += 1
```

```
def consumer():  
    global counter  
    for _ in range(ITERATIONS):  
        # Same here: Register1 = counter; Register1 = Register1 - 1; counter = Register1  
        counter -= 1
```

PRAVIN S. GAME

```
# 1. Create the threads  
producer_thread = threading.Thread(target=producer)  
consumer_thread = threading.Thread(target=consumer)
```

```
# 2. Start the race!  
producer_thread.start()  
consumer_thread.start()
```

```
# 3. Wait for both threads to finish before printing the result  
producer_thread.join()  
consumer_thread.join()
```

```
print(f"Expected counter value: 0")  
print(f"Actual counter value: {counter}")
```

PRAVIN S. GAME


```
@author: Radha Govind
"""

# Online Python compiler (interpreter) to run Python online.
# Write Python 3 code in this online editor and run it.
import threading

# Our shared buffer counter -- Critical section
counter = 0

# We use a large number to force the CPU to switch contexts
# while the threads are running.
ITERATIONS = 1000000

def producer():
    global counter
    for _ in range(ITERATIONS):
        # This looks like one step, but internally three steps.. Remember!
        counter += 1

def consumer():
    global counter
    for _ in range(ITERATIONS):
        # Same here: Register1 = counter; Register1 = Register1 - 1; counter -= 1
        counter -= 1

# 1. Create the threads
producer_thread = threading.Thread(target=producer)
consumer_thread = threading.Thread(target=consumer)

# 2. Start the race!
producer_thread.start()
consumer_thread.start()

# 3. Wait for both threads to finish before printing the result
producer_thread.join()
consumer_thread.join()

print(f"Expected counter value: 0")
print(f"Actual counter value: {counter}")
```

Console 2/A

```
Python 3.7.6 (default, Jan 8 2020, 20:23:39) [MSC v.1916 64 bit (AMD64)]
Type "copyright", "credits" or "license" for more information.

IPython 7.12.0 -- An enhanced Interactive Python.

In [1]: runfile('C:/Users/Radha Govind/PC problem.py', wdir='C:/Users/Radha Govind')
Expected counter value: 0
Actual counter value: -526527

In [2]: runfile('C:/Users/Radha Govind/PC problem.py', wdir='C:/Users/Radha Govind')
Expected counter value: 0
Actual counter value: 81637

In [3]: runfile('C:/Users/Radha Govind/PC problem.py', wdir='C:/Users/Radha Govind')
Expected counter value: 0
Actual counter value: -245231

In [4]: runfile('C:/Users/Radha Govind/PC problem.py', wdir='C:/Users/Radha Govind')
Expected counter value: 0
Actual counter value: 241066

In [5]: runfile('C:/Users/Radha Govind/PC problem.py', wdir='C:/Users/Radha Govind')
Expected counter value: 0
Actual counter value: -468150

In [6]: runfile('C:/Users/Radha Govind/PC problem.py', wdir='C:/Users/Radha Govind')
Expected counter value: 0
Actual counter value: 483580

In [7]:
```

Solution using semaphore

Shared data structures:

```
int n;
semaphore mutex = 1;
semaphore empty = n;
semaphore full = 0
```


Producer code:

```
while (true)
{
...
/*produce an item in next produced */
...
wait(empty);
wait(mutex);
...
/* add next produced to the buffer */
...
signal(mutex);
signal(full);
}
```

Consumer code:

```
while (true)
{
wait(full);
wait(mutex);
...
/*remove an item from buffer to next consumed */
...
signal(mutex);
signal(empty);
...
/* consume the item in next consumed */
...
}
```

THE DINING-PHILOSOPHERS PROBLEM

- Philosophers
 - Thinking
 - Eating
- Simple representation of the need
 - To allocate several resources among several processes
- Avoid deadlock and starvation



Solution using Semaphore

- Represent each chopstick by a semaphore
- Philosopher acquires chopstick – `wait()` on corresponding semaphore
- To release – `signal()` on that semaphore
- So, if there are 5 philosophers, shared data are
`semaphore chopstick[5]; // initialized to 1`
- Guarantee: No two neighbors are eating simultaneously

PRAVIN S. GAME

- Possible solutions to avoid deadlock:

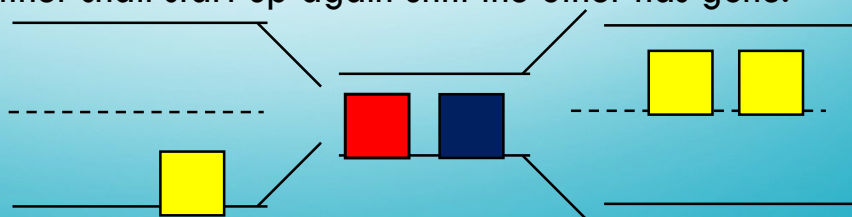
- i. Allow at most four ($n-1$) philosophers to be sitting simultaneously at the table
- ii. Allow a philosopher to pick up her chopsticks only if both chopsticks are available
- iii. Use an asymmetric solution

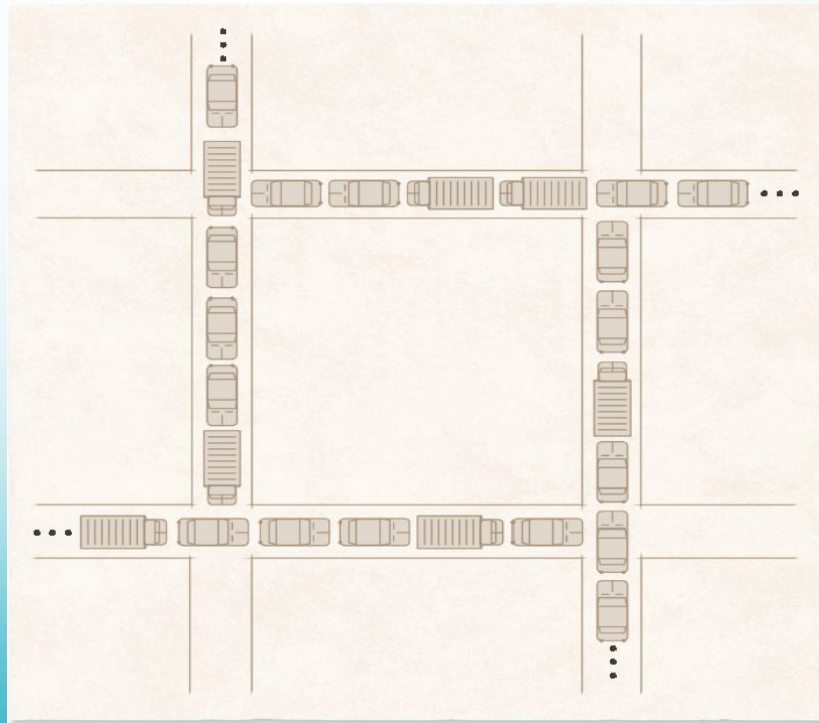
PRAVIN S. GAME

Deadlocks

DEFINITION

- Every process in a set of processes is waiting for an event that can be caused only by another process in the set.
- A set of blocked processes each holding a resource and waiting to acquire a resource held by another process in the set
- When two trains approach each other at a crossing, both shall come to a full stop and neither shall start up again until the other has gone.





PRAVIN S. GAME

SYSTEM MODEL

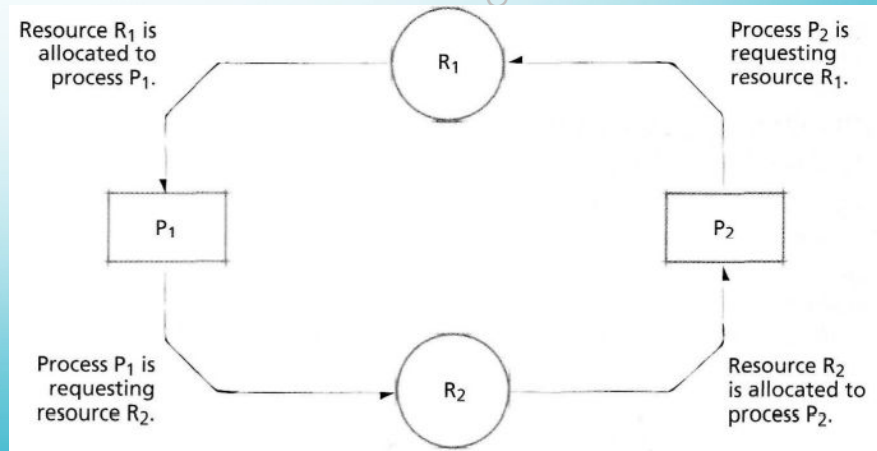
- Finite resources – R of type i : R_i
- Number of resources : W_i
- Resources: CPU cycles, files, memory, I/O devices, synchronization tools, resource from other process
- For a resource, each thread/process:
 - Must **request** before use
 - **Use** it
 - Must **release** after using
- Corresponding system calls may be made
- System maintains a resource allocation table

PRAVIN S. GAME

DEADLOCK CHARACTERIZATION

Four necessary conditions should hold simultaneously

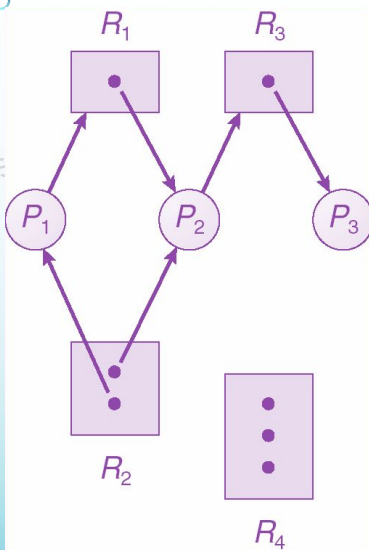
- Mutual exclusion
- Hold and wait
- No pre-emption
- Circular wait
- **Resource allocation graph**



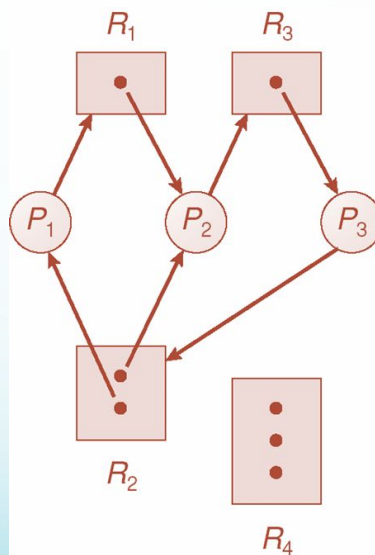
PRAVIN S. GAME

- RAG is a DIRECTED graph $G = \{V, E\}$
- V has two types: Process and Resource nodes
 - $P = \{P_1, P_2, \dots, P_n\}$, the set consisting of all the processes in the system
 - $R = \{R_1, R_2, \dots, R_m\}$, the set consisting of all resource types in the system
- **Request edge** – $P_i \rightarrow R_j$
- **Assignment edge** – $R_j \rightarrow P_i$
- Sets of P, R and E
- Resource instances
- Process states

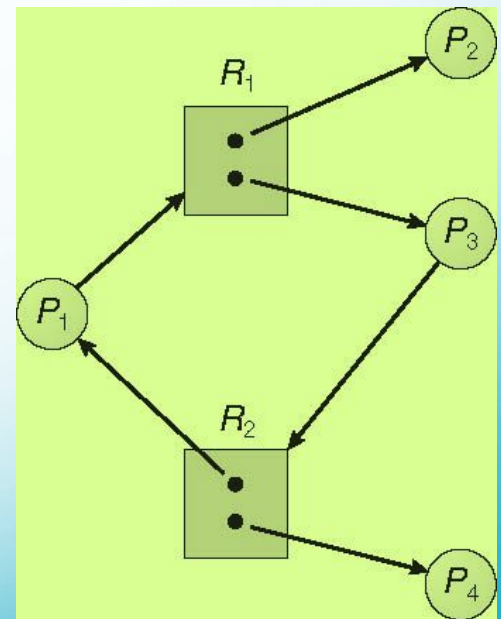
PRAVIN S. GAME



RAG



RAG with cycle
and deadlock



RAG with cycle but
NO deadlock

PRAVIN S. GAME

METHODS FOR HANDLING DEADLOCKS

1. Ignore and pretend that deadlock will never occur
2. Prevent or avoid so that system will never enter deadlock
3. Let system enter deadlock, detect and recover

Deadlock prevention - ensures that at least one of the necessary conditions cannot hold

Deadlock avoidance – gives OS additional information in advance for the required resources during the lifetime of the process.

PRAVIN S. GAME

DEADLOCK PREVENTION

Mutual exclusion

- Must hold
- Sharable resources do not require ME access – so no deadlock. (Read-only file)
- At least one resource must be nonsharable. (mutex lock)

Hold and wait

- Release all resources before requesting new – no hold
1. Process should request all required resources before starting execution
 2. Process is allocated a resource when it has none

•Disadv:

- Low resource utilization
- Starvation

PRAVIN S. GAME

No Preemption

1. Preempt currently held resources, if the requested resource is not available -> add into resource pool -> process gets resources only when old as well as requested resource are available
2. A. Check availability of requested resources(R_x) , if yes – allocate
B. if not, check if it is allocated to other process, which is waiting for resources. If yes, pre-empt required R_x & allocate to requesting process.
C. If R_x not available & not held by waiting process, requesting process goes in waiting state. (B) may occur

PRAVIN S. GAME

Circular wait

- First three options are impractical
- Practical solution – invalidate this condition

1. Order resource types
2. Request in increasing order

PRAVIN S. GAME

DEADLOCK AVOIDANCE

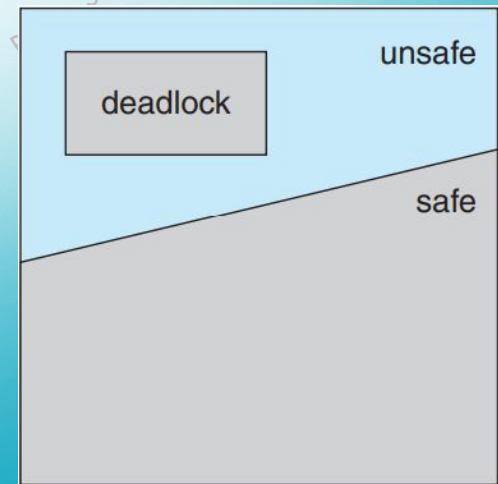
- Requires additional information
- P needs R1 and R2; Q needs R2 and then R1.
- Simplest method: each process to declare maximum no. of required resources.
- Algorithm dynamically check resource-allocate state to ensure that circular-wait condition never occurs
- **Safe State:**
System can allocate resources to each process in some order and still avoid a deadlock.

PRAVIN S. GAME

- A system is in a safe state only if there exists a **safe sequence**.
- There exists a sequence $\langle P_1, P_2, \dots, P_n \rangle$ of ALL the processes in the system such that for each P_i , the resources that P_i can still request can be satisfied by currently available resources + resources held by all the P_j , with $j < i$.

Process	Allocated resources	Current requirement	Max requirement
P1	5	5	10
P2	2	2	4
P3	2	2	9

- Safe sequence = $\langle P2, P1, P3 \rangle$



PRAVIN S. GAME

- So, avoidance just requires to ensure that system is in safe state.
- Two variants of avoidance algorithm

1. System with only one instance of each resource type – **RAG**
2. Multiple instances of each resource type – **Banker's algorithm**

PRAVIN S. GAME

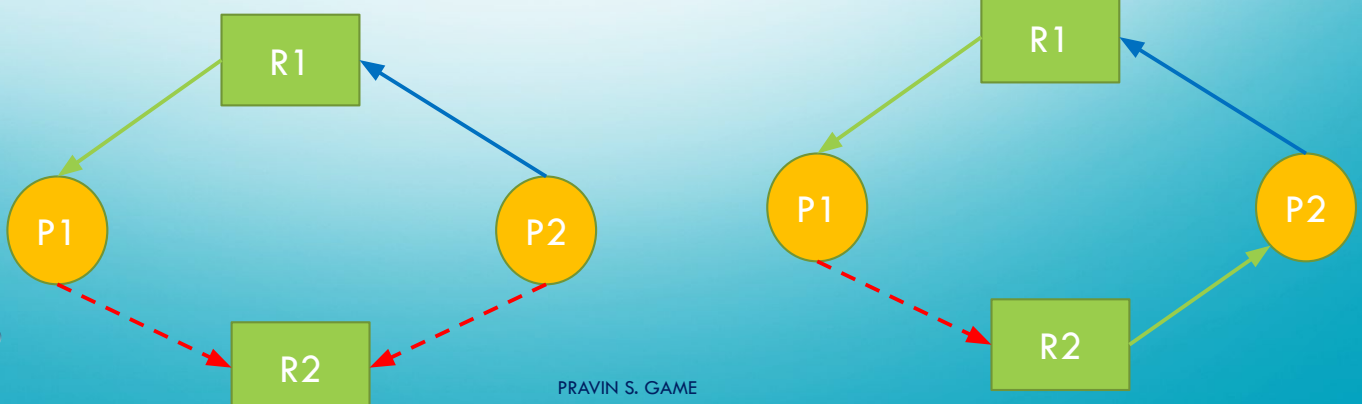
RAG

- **Request edge** – $P_i \rightarrow R_j$
- **Assignment edge** – $R_j \rightarrow P_i$
- **Claim edge** – $P_i \rightarrow R_j$
 - process P_i may request resource R_j ; represented by a **dashed line**
- Claim edge converts to request edge when a process requests a resource
- Request edge converted to an assignment edge when the resource is allocated to the process
- When a resource is released by a process, assignment edge reconverts to a claim edge

PRAVIN S. GAME

To avoid deadlock

- Resources must be claimed *a priori* in the system
- Request for a resource can be granted only if, converting the request edge $P_i \rightarrow R_j$ to an assignment edge $R_j \rightarrow P_i$ does not result in the formation of a cycle
 - No cycle : safe state
 - Cycle : unsafe state $\Rightarrow$ let process wait



PRAVIN S. GAME

BANKER'S ALGORITHM

PRAVIN S. GAME

BANKER'S ALGORITHM

- For multiple instances of resource types
 - Less efficient than RAG
 - Bank should never allocate its available cash in way that needs of all customers are not satisfied.
1. A new process enters system with list of required resources
 2. Number should not increase the maximum number of resources
 3. System determines whether allocation leads to safe state
 4. If yes, allocate resources
 5. If not, let process wait for resources to be freed

PRAVIN S. GAME

Required data structures: (n - processes; m – resource types)

- **Available:** Vector of length m . If available $[j] = k$, there are k instances of resource type R_j available
- **Max:** $n \times m$ matrix. If $Max[i, j] = k$, then process P_i may request at most k instances of resource type R_j
- **Allocation:** $n \times m$ matrix. If $Allocation[i, j] = k$ then P_i is currently allocated k instances of R_j
- **Need:** $n \times m$ matrix. If $Need[i, j] = k$, then P_i may need k more instances of R_j to complete its task

$$Need[i, j] = Max[i, j] - Allocation[i, j]$$

PRAVIN S. GAME

SAFETY ALGORITHM

1. Let *Work* and *Finish* be vectors of length m and n , respectively.
Initialize:

$$Work = Available$$

$$Finish[i] = false \text{ for } i = 0, 1, \dots, n-1$$

2. Find an index i such that both:

(a) $Finish[i] = false$

(b) $Need_i \leq Work$

If no such i exists, go to step 4

3. $Work = Work + Allocation_i$
 $Finish[i] = true$
go to step 2

4. If $Finish[i] == true$ for all i , then the system is in a safe state

PRAVIN S. GAME

RESOURCE-REQUEST ALGORITHM

- Let $Request_i$ be the request vector for process P_i
- If $Request_i[j] == k$, then process P_i wants k instances of resource type R_j

1. If $Request_i \leq Need_i$, go to step 2; else error.
2. If $Request_i \leq Available$, go to step 3; else P_i must wait
3. Pretend to allocate requested resources to P_i by modifying the state as follows:

$Available = Available - Request_i$;

$Allocation_i = Allocation_i + Request_i$;

$Need_i = Need_i - Request_i$;

If resultant allocation state is

- safe $\Rightarrow$ the resources are allocated to P_i
- unsafe $\Rightarrow P_i$ must wait, and the old resource-allocation state is restored

PRAVIN S. GAME

EXAMPLE

- Five process P_0 to P_4 ; Three resource types $A = 10$, $B = 5$ and $C = 7$ instances
- Current state

Process	Allocation	Max	Available
	$A - B - C$	$A - B - C$	$A - B - C$
P0	0 - 1 - 0	7 - 5 - 3	3 - 3 - 2
P1	2 - 0 - 0	3 - 2 - 2	
P2	3 - 0 - 2	9 - 0 - 2	
P3	2 - 1 - 1	2 - 2 - 2	
P4	0 - 0 - 2	4 - 3 - 3	

- Let's find Need matrix; $Need = Max - Allocation$

PRAVIN S. GAME

Process	Need		
	A	B	C
P0	7	4	3
P1	1	2	2
P2	6	0	0
P3	0	1	1
P4	4	3	1

- Is this system in safe state?
- Assume P1, submits additional Request₁(1,0,2)
- Step 1 - If $Request_i \leq Need_i$, go to step 2 : $(1,0,2) \leq (1,2,2)$?
- Step 2 - if $Request_i \leq Available$, go to step 3 : $(1,0,2) \leq (3,2,2)$?
- Step 3 – Pretend allocation

PRAVIN S. GAME

	<u>Allocation</u>			<u>Need</u>			<u>Available</u>		
	A	B	C	A	B	C	A	B	C
P ₀	0	1	0	7	4	3	2	3	0
P ₁	3	0	2	<u>0</u>	<u>2</u>	<u>0</u>			
P ₂	3	0	2	6	0	0			
P ₃	2	1	1	0	1	1			
P ₄	0	0	2	4	3	1			

- Now check whether system is in safe state?

PRAVIN S. GAME

Consider a system with five processes P0 through P4 and four resource types A, B, C, D. The current state of system is as follows:

Allocation matrix					Max allocation					Available			
	A	B	C	D		A	B	C	D	A	B	C	D
P0	0	0	1	2	P0	0	0	1	2	1	5	2	0
P1	1	0	0	0	P1	1	7	5	0				
P2	1	3	5	4	P2	2	3	5	6				
P3	0	6	3	2	P3	0	6	5	2				
P4	0	0	1	4	P4	0	6	5	6				

Identify whether the system is in deadlock by using Banker's algorithm.

[Step-by-step illustration is must]

Consider a system with five processes T0 through T4 and four resource types A, B, C, D. The current state of system is as follows:

	<u>Allocation</u>				<u>Max</u>				Available [0,3,0,1]
	A	B	C	D	A	B	C	D	
T ₀	3	0	1	4	5	1	1	7	
T ₁	2	2	1	0	3	2	1	1	
T ₂	3	1	2	1	3	3	2	1	
T ₃	0	5	1	0	4	6	1	2	
T ₄	4	2	1	2	6	3	2	5	

Identify whether the system is in deadlock by using Banker's algorithm.

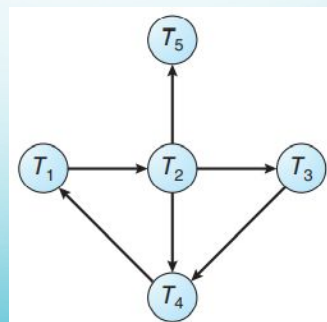
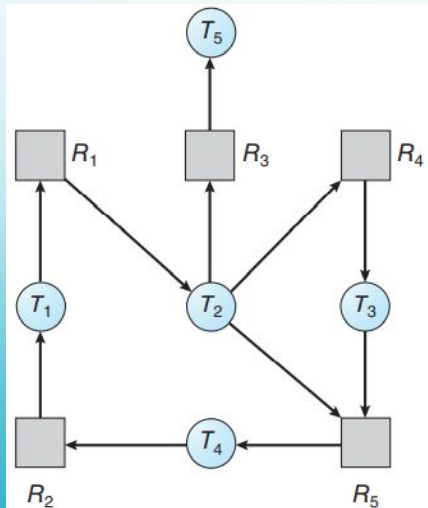
DEADLOCK DETECTION

ALLOW TO ENTER DEADLOCK, DETECT, RECOVER

PRAVIN S. GAME

SINGLE INSTANCE OF EACH RESOURCE TYPE

- RAG converted to wait-for graph
- Remove the resource nodes and collapse the appropriate edges



- Check for cycle
- If exists, deadlock

PRAVIN S. GAME

SEVERAL INSTANCES OF A RESOURCE TYPE

- **Available**
- **Allocation:** An $n \times m$ matrix
- **Request:** An $n \times m$ matrix indicates the current request of each process. If $Request[i][j] = k$, then process P_i is requesting k more instances of resource type, R_j .

PRAVIN S. GAME

DETECTION ALGORITHM

Let *Work* and *Finish* be vectors of length m and n , respectively

Initialize:

- $Work = Available$
- For $i = 1, 2, \dots, n$, if $Allocation_i \neq 0$, then
 $Finish[i] = false$; otherwise, $Finish[i] = true$

2. Find an index i such that both:

- $Finish[i] == false$
- $Request_i \leq Work$

If no such i exists, go to step 4

PRAVIN S. GAME

3. $Work = Work + Allocation_i$
 $Finish[i] = true$
go to step 2

4. If $Finish[i] == false$, for some i , $1 \leq i \leq n$, then the system is in deadlock state. Moreover, if $Finish[i] == false$, then P_i is deadlocked

PRAVIN S. GAME

RECOVERY FROM DEADLOCK: PROCESS TERMINATION

- Abort all deadlocked processes
- Abort one process at a time until the deadlock cycle is eliminated
- Order of abort....
 - Priority of the process
 - How long process has computed, and how much longer to completion
 - Resources the process has used
 - Resources process needs to complete
 - How many processes will need to be terminated
 - Is process interactive or batch?

PRAVIN S. GAME

RECOVERY FROM DEADLOCK: RESOURCE PREEMPTION

- Selecting a victim – minimize cost
- Rollback – return to some safe state, restart process for that state
- Starvation – same process picked as victim,
include number of rollback in cost factor

PRAVIN S. GAME

REFERENCES

- Silberschatz, Galvin, Gagne, "Operating System Principles", 10th Edition
- Stallings W., "Operating Systems- internals and design principles", 9th Edition.
- Deitel, Deitel, Choffnes, "Operating System", 3rd edition, Pearson
- Andrew S. Tanenbaum; Modern Operating Systems; Prentice Hall of India Publication; 4th Edition
- For POSIX - <https://www.open-std.org/jtc1/sc22/open/n4217.pdf> and <https://nvlpubs.nist.gov/nistpubs/Legacy/FIPS/fipspub151-1.pdf>

PRAVIN S. GAME

POSIX API FOR IPC

- Shared memory is organised using memory-mapped files

1. Create shared memory object

```
fd = shm_open(name, O_CREAT | O_RDWR, 0666)
```

- name : name of the shared-memory object
- O_CREAT : create shared memory object, if does not exist
- O_RDWR : object is open for R & W
- 0666 : file access permission
- fd : file descriptor returned on successful shm_open system call

PRAVIN S. GAME

2. Configure size of object in bytes

```
ftruncate(fd, 4096)
```

3. Map object to a file

```
ptr=mmap(0, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0)
```

- ptr : pointer to memory mapped file contained the object
- MAP_SHARED : flag specifies that changes are visible to all sharing processes

```
{ format pa=mmap(addr, len, prot, flags, fd, off);}
```

PRAVIN S. GAME